Efficient LLMs:
Principles and compression techniques

Ilya Boytsov

Senior ML Engineer, Nebius



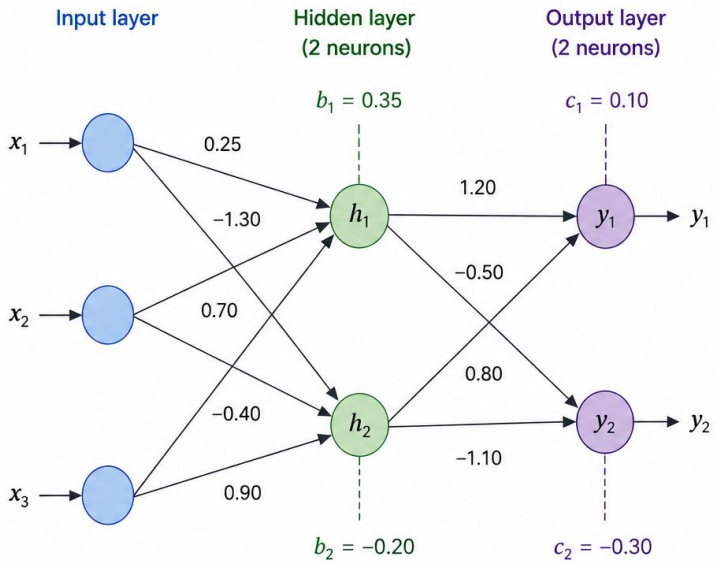
Oxford LLMs

Agenda

- How models store knowledge
- Why can we compress models with preserving most of the knowledge
- Compression algorithms
- Joint compression discussion
- Overview of related open-source tools
- Compressed models evaluations

How models store knowledge

Parameters in a Neural Network



Weights (parameters)

Input → Hidden ($W^{(1)}$)

	h_1	h_2
x_1	0.25	−1.30
x_2	0.70	0.10
x_3	−0.40	0.90

Hidden → Output ($W^{(2)}$)

	y_1	y_2
h_1	1.20	−0.50
h_2	0.80	−1.10

Biases (parameters)

Hidden layer bias (b)

h_1	h_2
0.35	−0.20

Output layer bias (c)

y_1	y_2
0.10	−0.30

Every number shown above (weights and biases) is a parameter.
A real LLM contains billions of such numbers, stored as floating-point values (e.g., FP16, BF16, FP32).

How much memory an LLM require?

Llama 3 8B

- 8 billion parameters
- Stored as FP16 (16 bits/parameter)

How much memory an LLM require?

Llama 3 8B

- 8 billion parameters
- Stored as FP16 (16 bits/parameter)

Memory = Parameters × Bits per Parameter

$8 \times 10^9 \times 16$ bits

⬇

128×10^9 bits

⬇

16 GB

Same model with different data precision

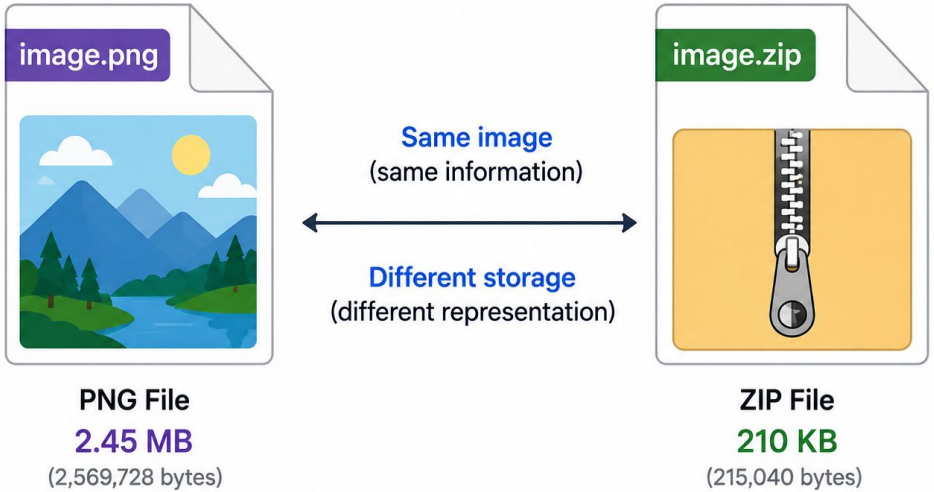
Model	Parameters	Precision	Bits / Parameter	Storage
Llama 3 70B	70B	FP16	16	140 GB
Llama 3 70B	70B	INT8	8	70 GB

If we reduce storage from 140 GB to 70 GB, have we lost 50% of the model's knowledge?

What we covered so far

- LLMs store knowledge in parameters
- Parameters are real numbers
- Changing the precision changes the memory footprint
- Yet the model still works

Storage ≠ Information



The number of stored bits is not necessarily equal to the amount of information.

Why can we compress models?

Why Can We Compress LLMs?

- Model can be overparameterized
- Some model weights can be close to zero
- Some model weights can correlate with each other
- low-rank structure in weight matrices
- Other

Quantized Qwen models accuracy

	Quantization	Average	MMLU	C-Eval	IFEval
Qwen2-72B-Instruct	BF16	81.3	82.3	83.8	77.6
	GPTQ-Int8	80.7	81.3	83.4	77.5
	GPTQ-Int4	81.2	80.8	83.9	78.9
	AWQ	80.4	80.5	83.9	76.9
Qwen2-7B-Instruct	BF16	66.9	70.5	77.2	53.1
	GPTQ-Int8	66.2	69.1	76.7	52.9
	GPTQ-Int4	64.1	67.8	75.2	49.4
	AWQ	64.1	67.4	73.6	51.4
Qwen2-1.5B-Instruct	BF16	48.4	52.4	63.8	29.0
	GPTQ-Int8	48.1	53.0	62.5	28.8
	GPTQ-Int4	45.0	50.7	57.4	27.0
	AWQ	46.5	51.6	58.1	29.9
Qwen2-0.5B-Instruct	BF16	34.4	37.9	45.2	20.0
	GPTQ-Int8	32.6	35.6	43.9	18.1
	GPTQ-Int4	29.7	33.0	39.2	16.8
	AWQ	31.1	34.4	42.1	16.7

Source: https://qwen.readthedocs.io/en/v2.5/benchmark/quantization_benchmark.html

Compression techniques

Compression techniques

- Quantization
- Pruning
- Low rank approximation
- Distillation

Quantization

Quantization is the process of representing values using fewer bits

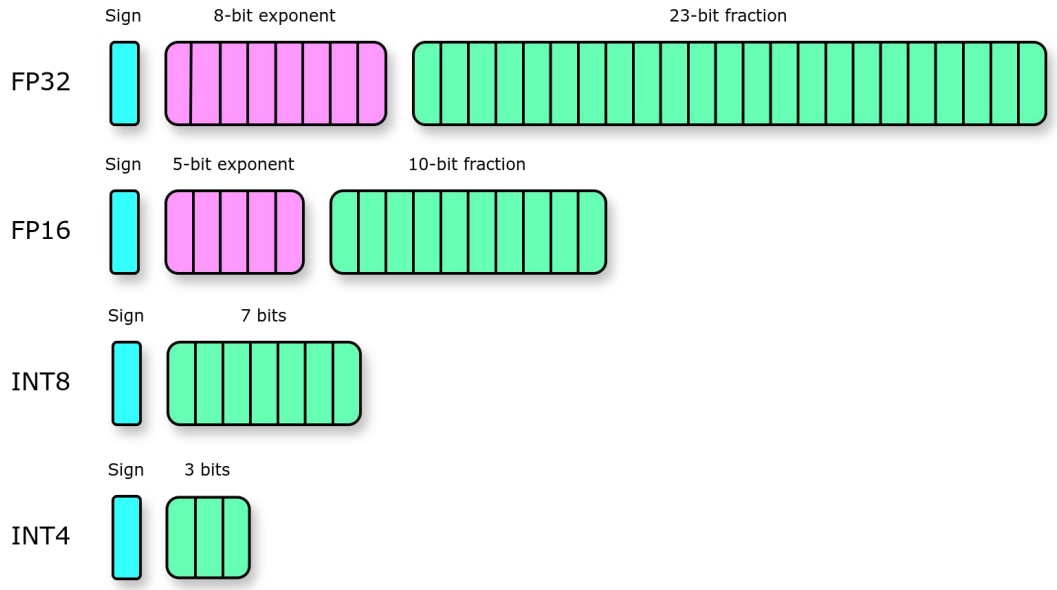
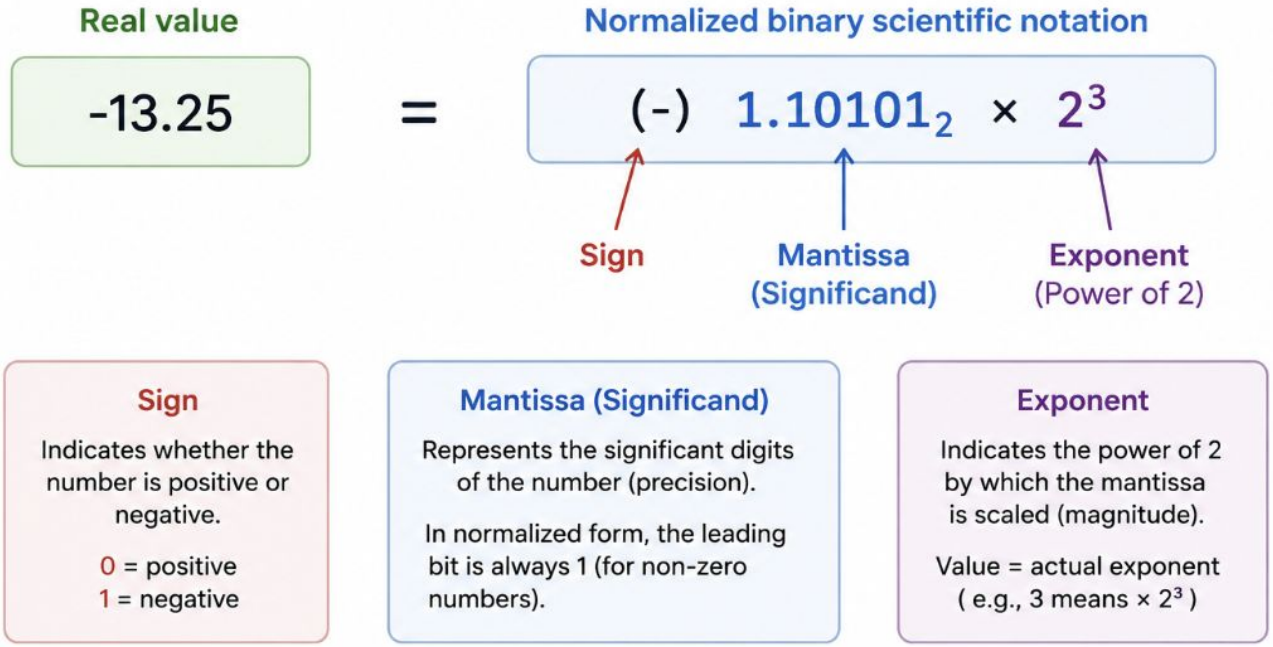


Image source: https://www.medoid.ai/blog/a-hands-on-walkthrough-on-model-quantization/

Binary number representation in FP32



Quantization Design Dimensions

Category	Types	Main Idea
When?	Post-Training Quantization (PTQ) Quantization-Aware Training (QAT)	Quantize the model after training or during training to improve accuracy.
What?	Weights, Activations, KV Cache	Choose which tensors in the model are quantized.
Precision	FP16, INT8, INT4, INT2, ...	Number of bits used to represent each value.
Granularity	Per-Tensor, Per-Channel, Per-Group	Decide how quantization parameters (scale/zero-point) are shared.
Mapping	Symmetric, Asymmetric	Define how floating-point values are mapped to integers.

Quantization Design Example Recipe

Category	Types	Main Idea
When?	Post-Training Quantization (PTQ) Quantization-Aware Training (QAT)	Quantize the model after training or during training to improve accuracy.
What?	Weights, Activations, KV Cache	Choose which tensors in the model are quantized.
Precision	FP16, INT8, INT4, INT2, ...	Number of bits used to represent each value.
Granularity	Per-Tensor, Per-Channel, Per-Group	Decide how quantization parameters (scale/zero-point) are shared.
Mapping	Symmetric, Asymmetric	Define how floating-point values are mapped to integers.

Original FP16	Stored INT8
-0.82	-82
-0.15	-15
0.384	38
0.917	92

Scale (s) = 0.01

- All weights in a tensor (or per-channel/per-group) share the same **scale** s.
- **Quantization:** $q = round(w / s)$
- **Dequantization** (conceptually): $\hat{w} = q \times s$

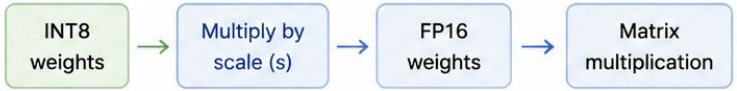
We store only the **INT8** integers and one **scale** value.
The integer alone is not meaningful.

To recover the approximate weight:

$$\hat{w} = q \times s$$

How it is used during inference

1. On hardware without INT8 support



2. On modern accelerators (GPUs, TPUs, CPUs with INT8)



- Integer matrix multiplication is performed directly.
- Scaling is applied after accumulation.
- The full FP16 weights are never reconstructed.

Quantization **reduces memory** by storing integers instead of floating-point numbers.
The **scale** lets us map integers back to approximate floating-point values when needed.

Choosing the Quantization Scale

Goal: Map floating-point values to INT8 range $[-128,\ 127]$ using a scale s.

1 Compute the dynamic range

Weights (FP32 / FP16)

−1.82	−0.91	−0.37	0.14	0.65	1.42

$$max(|W|) = \mathbf{1.82}$$

2 Choose the scale

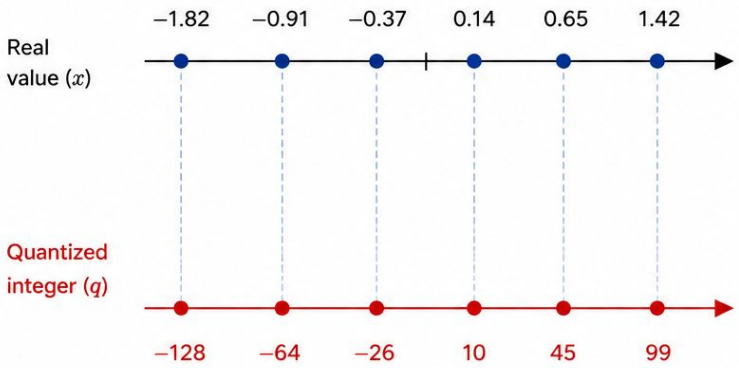
$$s = \frac{\max(|W|)}{127} = \frac{1.82}{127} \approx 0.01433$$

3 Quantize the values

$$q = round\left(\frac{x}{s}\right)$$

clip to $[-128, 127]$

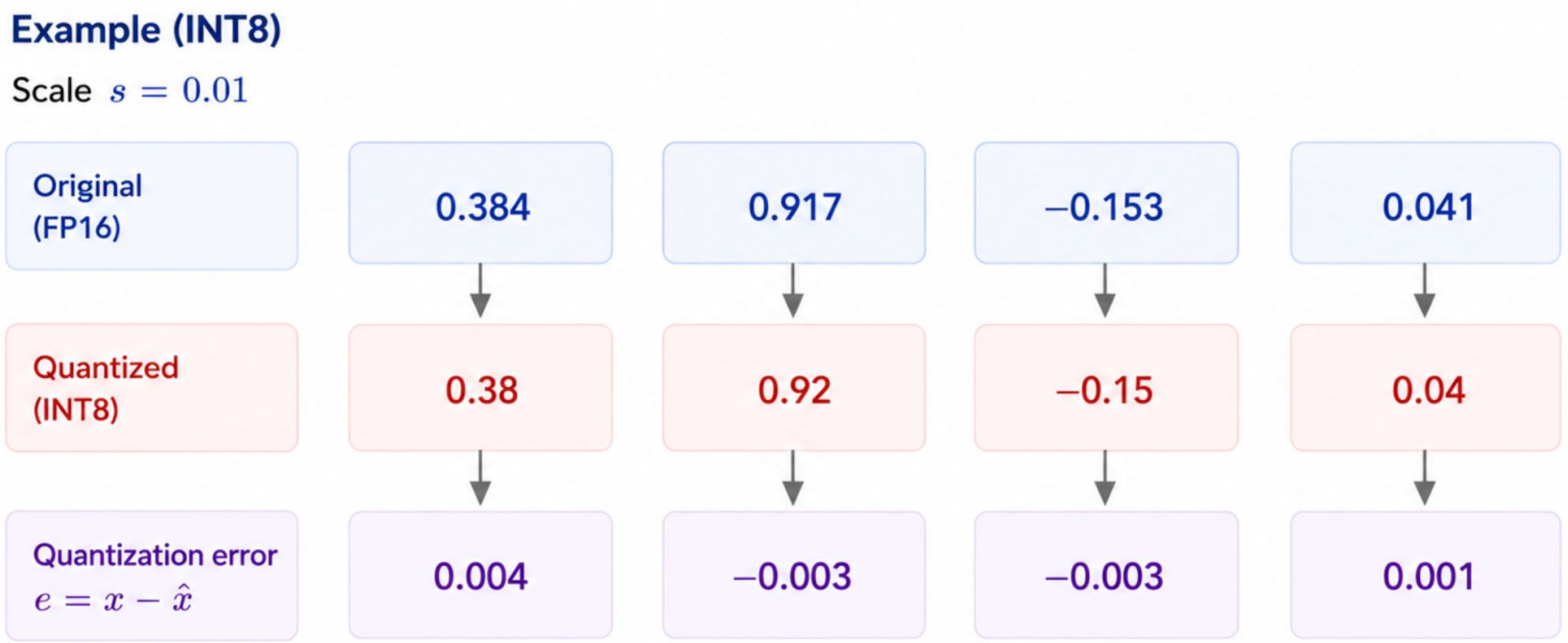
Mapping visualization



INT8 range: $[-128, 127]$ (255 levels)

Scale: $s \approx 0.01433$

Quantization error



Debugging Quantization

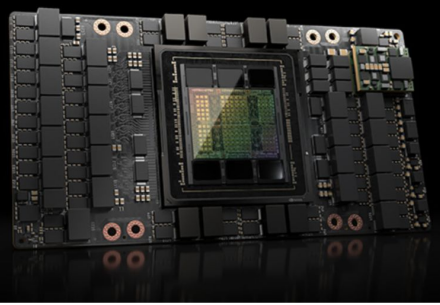
- Measure the accuracy drop.
- Quantize one layer at a time to identify sensitive layers.
- Inspect weight and activation distributions.
- Verify scale and zero-point are computed and applied correctly.
- Measure quantization error (e.g., MSE or cosine similarity).
- Try finer granularity (per-channel instead of per-tensor).
- Keep sensitive layers in higher precision (e.g., embeddings, output head, normalization layers).

Quantization and hardware

NVIDIA Hopper Architecture

The engine for the world's AI infrastructure makes an order-of-magnitude performance leap.

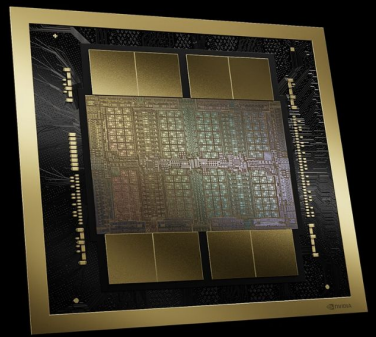
Learn More



NVIDIA Blackwell Architecture

The engine behind AI factories for the age of AI reasoning—now in full production.

Read Technical Brief



Practical considerations

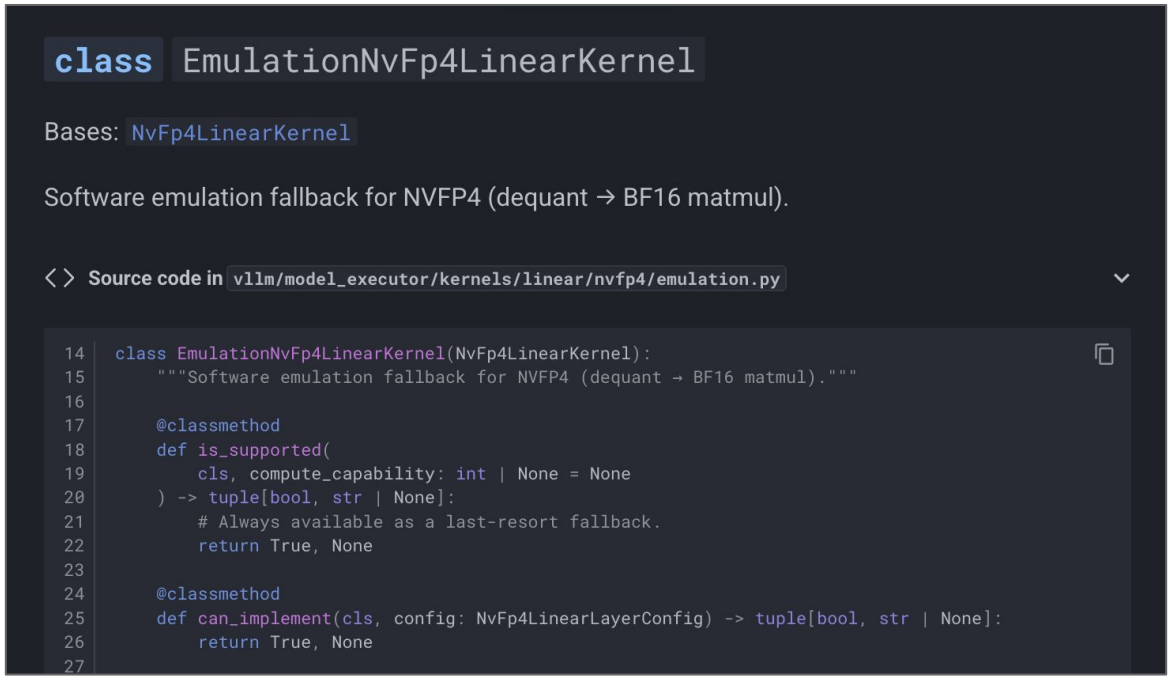
- Choose quantization based on the deployment hardware.
- Verify that the hardware natively supports the target precision.
- Lower precision reduces memory bandwidth requirements.
- Lower precision often increases throughput on supported hardware.
- Some quantization methods require specialized inference kernels.
- Memory savings do not always translate to speedups.
- Always benchmark on the target device.

vLLM case: quantization-hardware mismatch

What will happen if user runs nvidia/MiniMax-M2.5-NVFP4 on H100 GPUs?

vLLM case: quantization-hardware mismatch

What will happen if user runs nvidia/MiniMax-M2.5-NVFP4 on H100 GPUs?



class EmulationNvFp4LinearKernel

Bases: NvFp4LinearKernel

Software emulation fallback for NVFP4 (dequant → BF16 matmul).

Source code in vllm/model_executor/kernels/linear/nvfp4/emulation.py

```
14  class EmulationNvFp4LinearKernel(NvFp4LinearKernel):
15      """Software emulation fallback for NVFP4 (dequant → BF16 matmul)."""
16
17      @classmethod
18      def is_supported(
19          cls, compute_capability: int | None = None
20      ) -> tuple[bool, str | None]:
21          # Always available as a last-resort fallback.
22          return True, None
23
24      @classmethod
25      def can_implement(cls, config: NvFp4LinearLayerConfig) -> tuple[bool, str | None]:
26          return True, None
27
```

Source

Pruning

Pruning

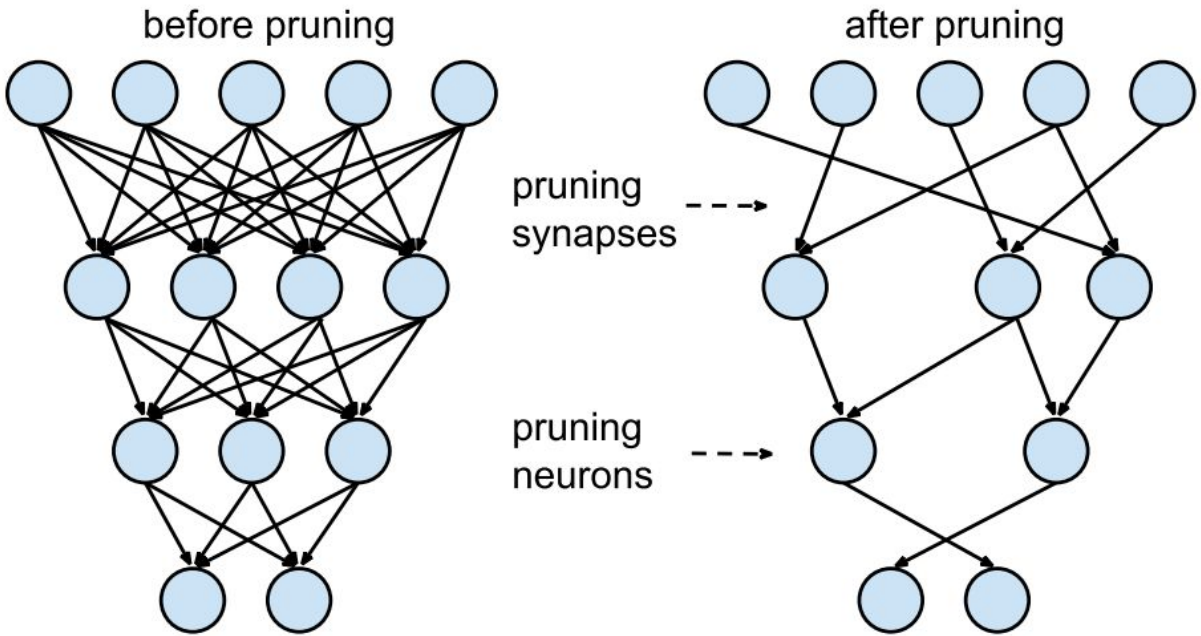
Quantization asks:

Can we use fewer bits per weight?

Pruning asks:

Do we need all of the weights at all?

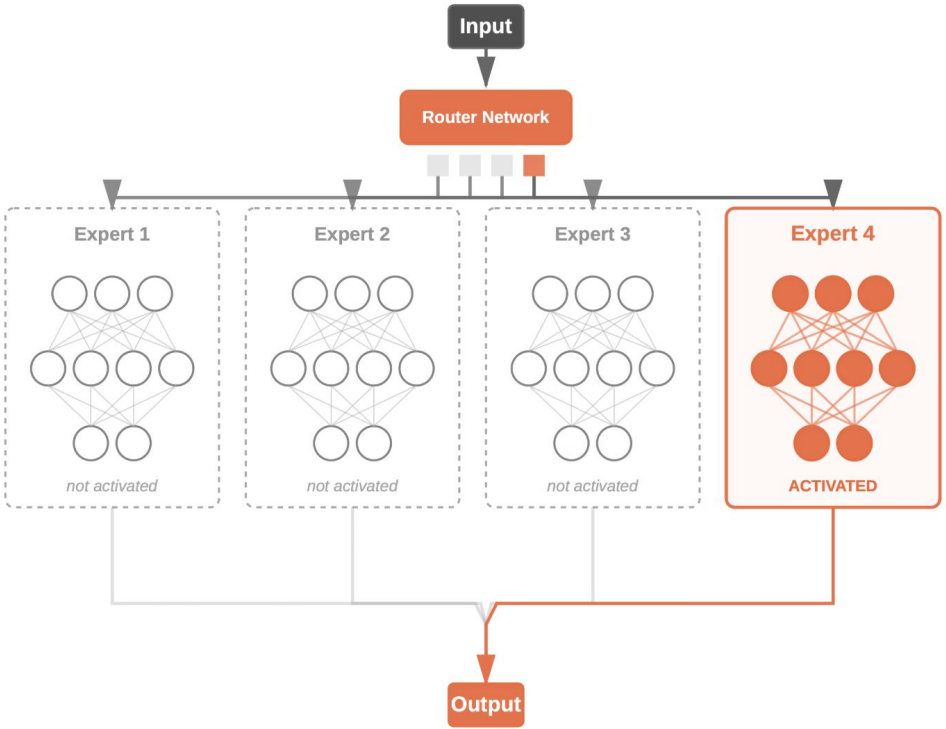
Pruning


before pruning
after pruning
pruning
synapses
pruning
neurons

Prerequisites: Mixture of Experts (MoE)

In this lecture we will talk about pruning MoE models because MoE is a standard architecture frontier models utilise in 2k26.

Basic MoE block

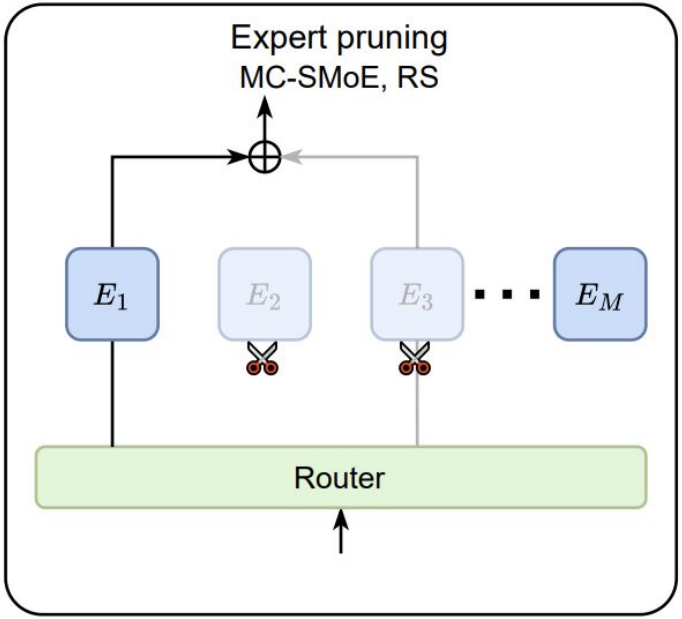


Source: https://www.cerebras.ai/blog/reap

Advantages of MoE

- Massive model capacity without proportionally increasing inference cost.
- Sparse activation - only a small subset of experts is used for each token.
- Faster than a dense model with a similar number of parameters.
- Experts can specialize in different domains, languages, or reasoning patterns.

Can we remove some experts?



Source: https://arxiv.org/pdf/2507.00390

How to identify redundant experts?

- Activation Frequency
- Output variance
- Output magnitude
- Router confidence

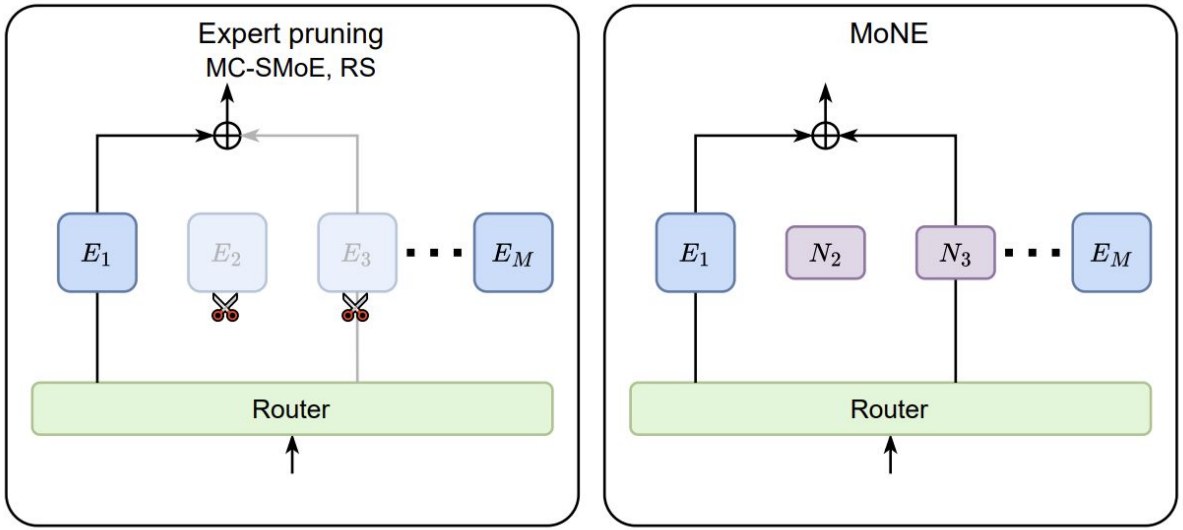
REAP: Router weighted Expert Activation Pruning

$$\text{REAP Score} = \underbrace{\frac{1}{|\mathcal{X}_j|}}_{\text{average over tokens routed to expert } j} \sum_{x \in X_j} \underbrace{g_j(x)}_{\text{router gate value (weight) for token } x} \underbrace{\|f_j(x)\|_2}_{\text{L2 norm of expert } j\text{'s output for token } x}$$

Source: https://www.cerebras.ai/blog/reap

MoNe: Mixture of Novices and Experts

Key idea: identify redundant experts but instead of removing them replace them with constant vectors that are expert's unbiased estimators



Source: https://arxiv.org/pdf/2507.00390

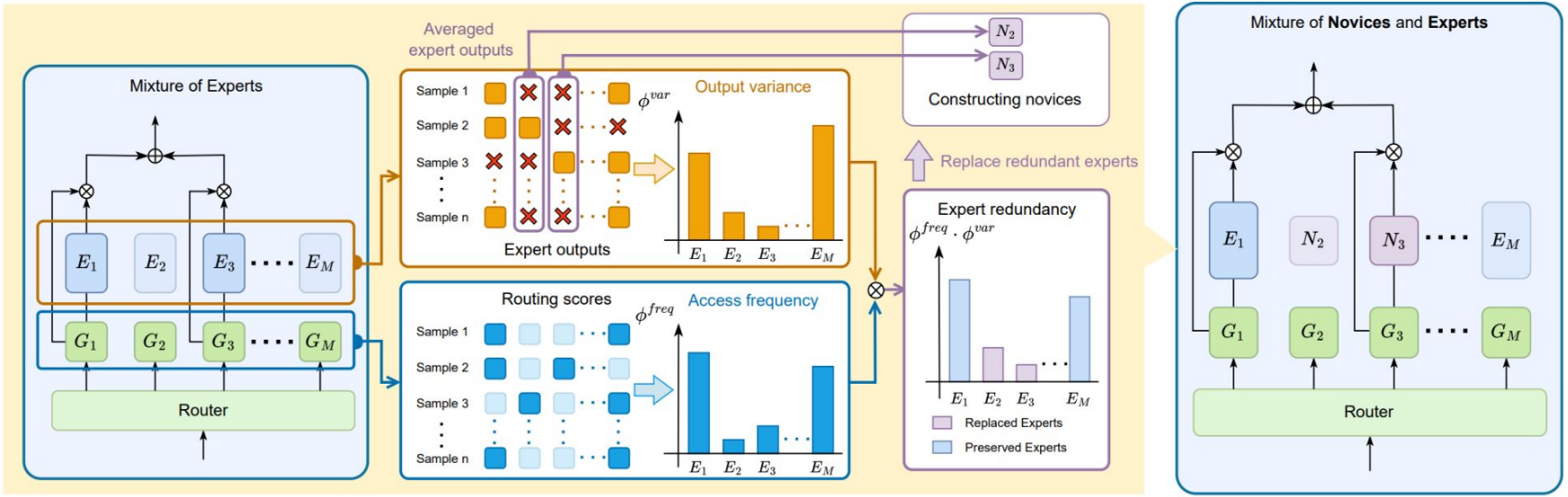


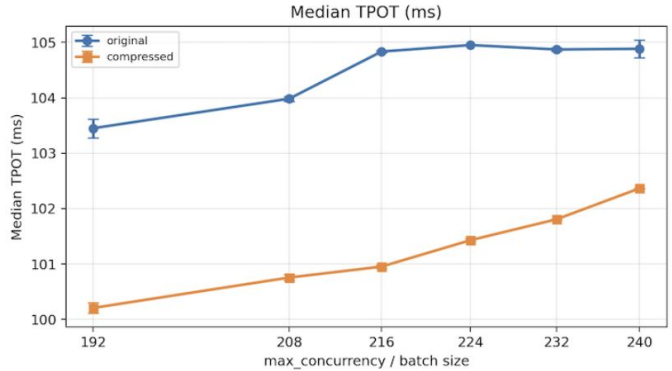
Figure 2: **The overview of MoNE.** Given an MoE model, it first exploits a calibration dataset to evaluate the expert access frequency and output variance. Then, the two metrics are fused to get the expert redundancy. Finally, the novices are derived from the averaged outputs for redundant experts.

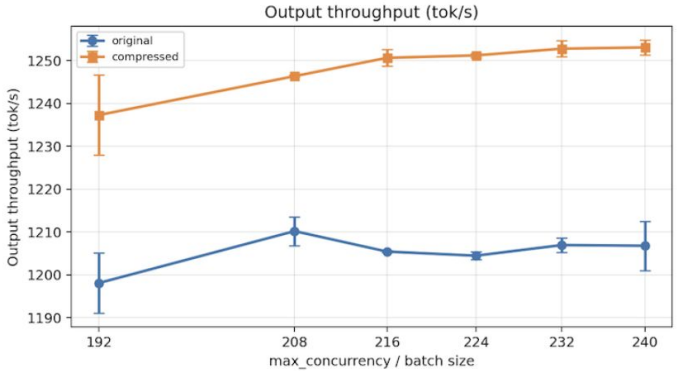
Source: https://arxiv.org/pdf/2507.00390

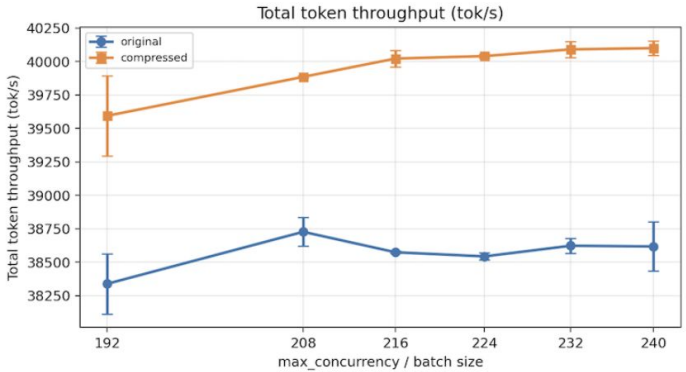
MoNe Advantages (if implemented correctly)

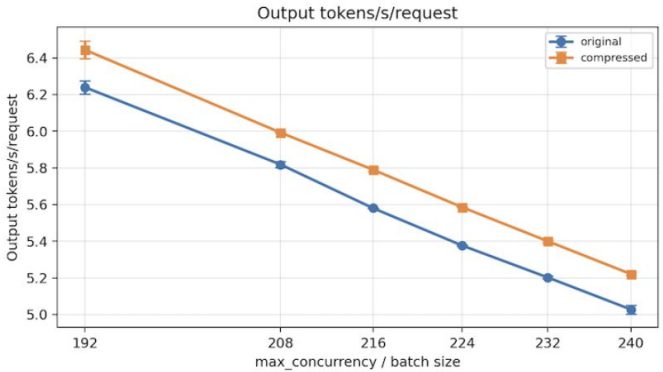
- GPU memory reduction (e.g. 25% of pruned experts lead to 23 % reduction)
- Compute speedup:
  - throughput
  - number of tokens per second

MoNe performance benchmarking







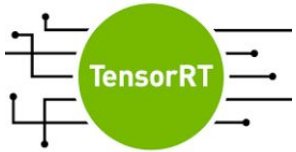


MoNe inference serving challenges

- MoNe modifies source model architecture. Frameworks can’t work with it as is.
- For performance impact MoNe requires custom CUDA kernels







Joint Compression

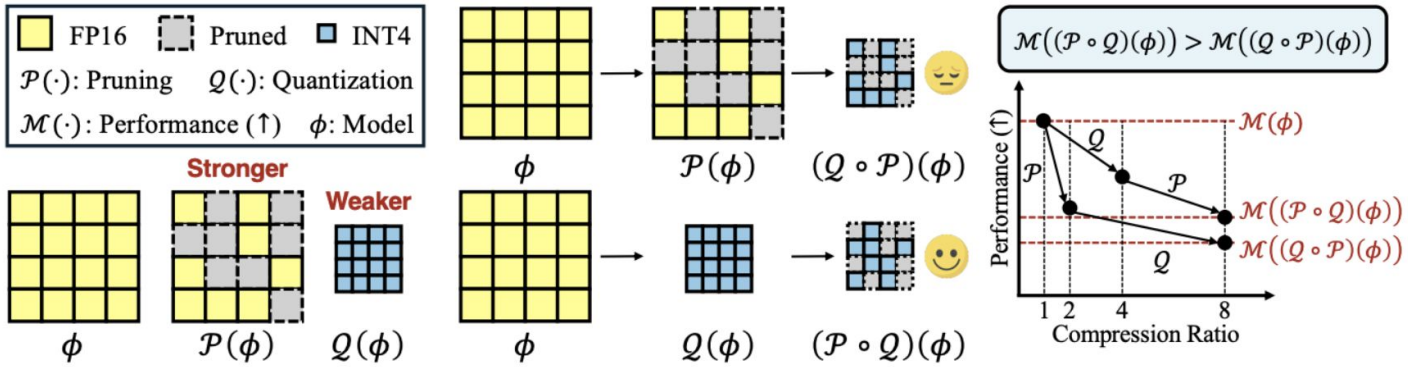
Stacking multiple compression algorithms

Central questions:

- How does joint compression impact accuracy?
- How to find an optimal combination to achieve accuracy-efficiency tradeoff?

Progressive Intensity Hypothesis

The Progressive Intensity Hypothesis. Neural networks compressed by multiple methods perform better when weaker perturbations are applied first and stronger ones later.



(a) Model compression techniques (b) Joint model compression scenarios (c) Model performance

Figure 1: **The Progressive Intensity Hypothesis:** Given two compression techniques, we conjecture that compressed models perform better if the stronger method is applied after the weaker one. That said, the optimal order between pruning and quantization varies with their compression ratios.

Compression Equivalent Ratio (CER)

CER maps any compression method to an equivalent quantization bit width based on the model's performance. Quantization is chosen as the reference because bitwidth provides a natural, continuous, and widely understood measure of compression severity.

How to compare the strength of different compression methods?

Quantization accuracy

Bitwidth	Accuracy
8-bit	82.0%
6-bit	81.7%
4-bit	80.8%
2-bit	76.2%

Pruning accuracy

Sparsity ↓	Accuracy
30%	81.9%
50%	81.1%
70%	79.3%

The accuracy of 50% pruning falls between that of 4-bit and 6-bit quantization.
Linear interpolation suggests its Compression Equivalent Ratio (CER) is about 4.7 bits.

Open-source model compression tools

Compression tools

https://github.com/NVIDIA/Model-Optimizer

https://github.com/vllm-project/llm-compressor

Compressed models evaluation

MiniMax-M2.5 FP8 vs NVFP4 benchmarking

Precision	MMLU Pro	GPQA Diamond	LiveCodeBench V6	SciCode	AIME 2025	AA-LCR	IFBench
FP8	0.825	0.845	0.583	0.453	0.869	0.676	0.734
NVFP4	0.822	0.839	0.577	0.452	0.853	0.674	0.752

> *"Baseline: MiniMax-M2.5. Benchmarked with temperature=1.0, top_p=0.95, max num tokens 64000"*

Source

Statistical approach to benchmarking

When LLMs Get Significantly Worse: A Statistical Approach to Detect Model Degradations

Statistical approach to model evaluations

Permutation test

Select a benchmark and run each model, collect per-prompt model scores.

Compute delta as Model_original_score - Model_optimised_score

Prompt_id	Model_original_score	Model_optimised_score	Delta
1	0.34	0.27	0.07
2	0.92	0.98	-0.06

Permutation test

Null hypothesis: original and optimized are equally good

Prompt_id	Model_original_score	Model_optimised_score	Delta
1	0.34	0.27	0.07
2	0.92	0.98	-0.06

Permutation test

Null hypothesis: original and optimized are equally good

Under that the labels of original and optimized models should be exchangeable within each prompt.

Prompt_id	Model_original_score	Model_optimised_score	Delta
1	0.34	0.27	0.07
2	0.92	0.98	-0.06

Permutation test

Null hypothesis: original and optimized are equally good

Under that the labels of original and optimized models should be exchangeable within each prompt.

Permutation test run N simulations where it randomly flips scores signs.

Prompt_id	Model_original_score	Model_optimised_score	Delta
1	0.34	0.27	0.07
2	0.92	0.98	-0.06

Permutation test

Null hypothesis: original and optimized are equally good

Under that the labels of original and optimized models should be exchangeable within each prompt.

Permutation test run N simulations where it randomly flips scores signs.

p value = fraction of permutations where simulated degradation >= observed degradation

Prompt_id	Model_original_score	Model_optimised_score	Delta
1	0.34	0.27	0.07
2	0.92	0.98	-0.06

Summary

- Compression methods boost model efficiency when aligned with hardware.
- LLM Compression is a lossy optimization, it may lead to accuracy drop.
- Compression requires extended evaluations to test whether model degrades.

Thank you!